

LABORATORY

LAB 13

Distance Monitor with TFT

Integrating HC-SR04, TFT display, and LED warning with
EduFramework

S32K144 · MaaZEDU

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	Integrating Functional Blocks	2
3	Hardware Setup	3
3.1	Required Hardware	3
3.2	Pin Mapping	3
3.3	Hardware Connections	3
4	EduFramework API	4
5	Lab Exercise	5
5.1	Requirements	5
5.2	Program	5
5.3	Program Explanation	6
5.4	Verification	8
6	Extension	8
7	References	8

1 Introduction

1.1 Lab Overview

Lab 09 used the HC-SR04 to measure distance, while Lab 12 introduced the TFT display and basic display APIs. This lab combines these previously learned components into a simple distance monitoring application.

The HC-SR04 provides distance data to the application. Valid measurements are displayed directly on the TFT in centimeters. When the measured distance is less than or equal to 5 cm, the on-board red LED on MaaZEDU blinks to provide a visual warning.

This lab does not introduce any new Device API. The focus is on coordinating multiple previously learned APIs within a single program to form a complete processing flow from data acquisition to display and warning indication.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Describe the processing flow from sensor data to display output and warning indication.
2. Combine the Ultrasonic Device API, TFT Device API, and Digital I/O API within a single application.
3. Update the measured distance on the TFT display in real time.
4. Implement and verify LED warning logic when the measured distance is less than or equal to 5 cm.

2 Background

2.1 Integrating Functional Blocks

In previous labs, sensors, displays, and Digital Output were used as separate functional blocks. In a more complete application, these blocks can be combined into a common processing flow:

Sensing → Processing → Presentation / Indication

In this lab, the HC-SR04 performs data acquisition. The application checks the measurement result and uses the same distance value for two purposes: displaying quantitative data on the TFT and determining the state of the warning LED.

The data flow can be represented as follows:

HC-SR04 → `ultrasonicRead()` → Application Logic → TFT + LED_RED

When a valid measurement is available, the distance is displayed on the TFT. If the value is less than or equal to 5 cm, the red LED is toggled during each update cycle to create a blinking effect. When the distance is above the threshold or the measurement is invalid, the LED is returned to the OFF state.

Note

The HC-SR04 distance measurement principle was covered in Lab 09. TFT coordinates, colors, and display operations were covered in Lab 12 and are therefore not repeated in this lab.

3 Hardware Setup

3.1 Required Hardware

This lab uses the HC-SR04 as the distance data source, the LCD TFT as the display device, and the on-board red LED on MaaZEDU as the warning indicator.

Hardware	Description
MaaZEDU Development Board	Development board based on the S32K144 microcontroller.
HC-SR04	Ultrasonic sensor used for distance measurement.
LCD TFT	Displays the measured distance value.
Jumper wires	Connect power and signal lines between the modules and MaaZEDU.
USB Cable	Connects the board to a computer for power and program uploading.

Table 13.3.1. Hardware used in this lab

3.2 Pin Mapping

In Lab 09, the HC-SR04 used `GPIO2` and `GPIO1`. These Logical Pins are now used by the default TFT Device configuration, so this lab moves the `TRIG` and `ECHO` signals to `GPIO4` and `GPIO5`.

According to the current MaaZEDU pin mapping, `GPIO4` corresponds to `PTD12`, `GPIO5` corresponds to `PTD11`, and the on-board red LED uses `PTD15` [3] [6].

Component	Logical Pin	MCU Pin	Function
HC-SR04 TRIG	<code>GPIO4</code>	<code>PTD12</code>	Digital Output
HC-SR04 ECHO	<code>GPIO5</code>	<code>PTD11</code>	Digital Input
On-board red LED	<code>LED_RED</code>	<code>PTD15</code>	Warning Output

Table 13.3.2. HC-SR04 and warning LED pin mapping

3.3 Hardware Connections

The HC-SR04 is powered from `5V` and shares `GND` with MaaZEDU. The `TRIG` signal is connected to `GPIO4`, while `ECHO` is connected to `GPIO5`. The power and `TRIG` / `ECHO` signal arrangement follows the basic HC-SR04 usage configuration [2] [1].

The TFT continues to use the default configuration introduced in Lab 12. The `CS`, `DC`, `RST`, and `BLK` signals use `GPIO0` through `GPIO3`; the SPI clock and data lines use `SPI_SCK` and `SPI_SOUT` [5].

Device / Pin	MaaZEDU	Function
HC-SR04 VCC	5V	Sensor power supply.
HC-SR04 GND	GND	Common ground.
HC-SR04 TRIG	GPIO4	Trigger signal.
HC-SR04 ECHO	GPIO5	Echo signal.
TFT VCC	3.3V	TFT power supply.
TFT GND	GND	Common ground.
TFT SCL / SCK	SPI_SCK	SPI clock.
TFT SDA / MOSI	SPI_SOUT	SPI data from the MCU to the TFT.
TFT CS	GPIO0	Chip Select.
TFT DC	GPIO1	Data / Command select.
TFT RES / RST	GPIO2	Hardware Reset.
TFT BLK / BL	GPIO3	Backlight control.

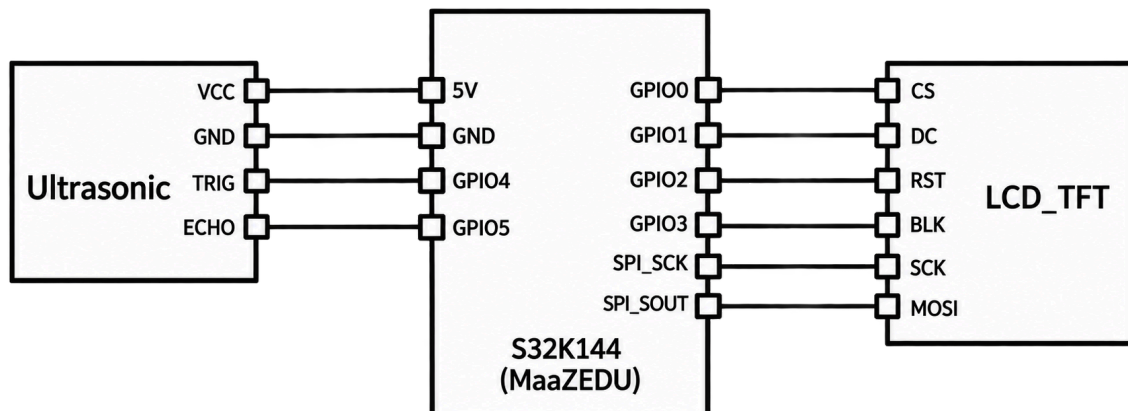


Figure 13.3.1. HC-SR04 and LCD TFT connections to the MaaZEDU Development Board

4 EduFramework API

Lab 13 does not introduce any new API. The program combines APIs previously used in Lab 01, Lab 09, and Lab 12 to form a complete monitoring application [4] [5] [6] .

Function	Main APIs	Introduced in
Distance measurement	<code>ultrasonicBegin()</code> , <code>ultrasonicRead()</code>	Lab 09
TFT display	<code>TFT_Begin()</code> , <code>TFT_FillRect()</code> , <code>TFT_PrintFloat()</code>	Lab 12
LED control	<code>pinMode()</code> , <code>digitalWrite()</code> , <code>digitalToggle()</code>	Lab 01
Update timing	<code>delay()</code>	Lab 01

Table 13.4.1. API groups combined in Lab 13

The main API usage flow in the application is:

`ultrasonicRead()` → Check Result → Update TFT → Control LED

The syntax and parameters of each API are not repeated in this lab.

5 Lab Exercise

5.1 Requirements

Build a distance monitoring program using the HC-SR04 and display the result on the TFT.

The program must meet the following requirements:

- Initialize `LED_RED`, the HC-SR04, and the TFT.
- Use `GPIO4` for `TRIG` and `GPIO5` for `ECHO`.
- Read and display valid distance measurements on the TFT in centimeters.
- If the measurement is invalid, display `--. - cm`.
- When the distance is greater than `5 cm`, keep `LED_RED` OFF.
- When the distance is less than or equal to `5 cm`, blink `LED_RED` as a warning.
- Update the measurement and warning state every `200 ms`.

5.2 Program

In `src/main.c`, implement the hardware-verified program as follows:

```
#include "Arduino.h"
#include "ultrasonic.h"
#include "lcd_tft.h"

int main(void)
{
    float distance = ULTRASONIC_INVALID_DISTANCE_CM;
    setup();
    pinMode(LED_RED, OUTPUT);
    digitalWrite(LED_RED, HIGH);
    ultrasonicBegin(GPIO4, GPIO5);
    TFT_Begin();
    TFT_FillScreen(TFT_BLACK);
    TFT_SetTextColor(TFT_WHITE);
    TFT_SetTextBackground(TFT_BLACK);
    TFT_SetTextSize(2U);
    TFT_SetCursor(55U, 40U);
    TFT_Print("DISTANCE");
    while (1)
    {
        distance = ultrasonicRead();
        TFT_FillRect(30U, 100U, 180U, 50U, TFT_BLACK);
        TFT_SetCursor(55U, 110U);
        TFT_SetTextSize(3U);
        if (ULTRASONIC_INVALID_DISTANCE_CM != distance)
        {
            TFT_PrintFloat(distance, 1U);
            TFT_Print(" cm");
            if (distance <= 5.0f)
            {
                digitalWrite(LED_RED);
            }
            else
            {
                digitalWrite(LED_RED, HIGH);
            }
        }
        else
        {
            TFT_Print("--.- cm");
            digitalWrite(LED_RED, HIGH);
        }
        delay(200U);
    }
    return 0;
}
```

Listing 13.5.1. HC-SR04 and TFT distance monitoring program

5.3 Program Explanation

After `setup()`, `LED_RED` is configured as a Digital Output and driven `HIGH` to keep the on-board red LED OFF. The HC-SR04 is initialized with `GPIO4` as `TRIG` and `GPIO5` as `ECHO`, after which the TFT is initialized and the initial screen layout is prepared.

The `DISTANCE` title is drawn only once before the main loop. During each cycle, `ultrasonicRead()` obtains a new measurement. Before the next value is shown, `TFT_FillRect()` clears only the area containing the previous result instead of clearing the entire screen. This keeps the title unchanged while allowing the application to update only the content that needs to change.

If the measurement is valid, `TFT_PrintFloat()` displays the distance with one digit after the decimal point. The value is then compared with the `5.0f` threshold. When the distance is less than or equal to the threshold, `digitalToggle(LED_RED)` changes the LED state during each update cycle. When the distance is greater than the threshold, `digitalWrite(LED_RED, HIGH)` returns the LED to the OFF state.

If `ultrasonicRead()` returns `ULTRASONIC_INVALID_DISTANCE_CM`, the program displays `--.- cm` and keeps the LED OFF [4].

The overall processing flow is shown in the following diagram:

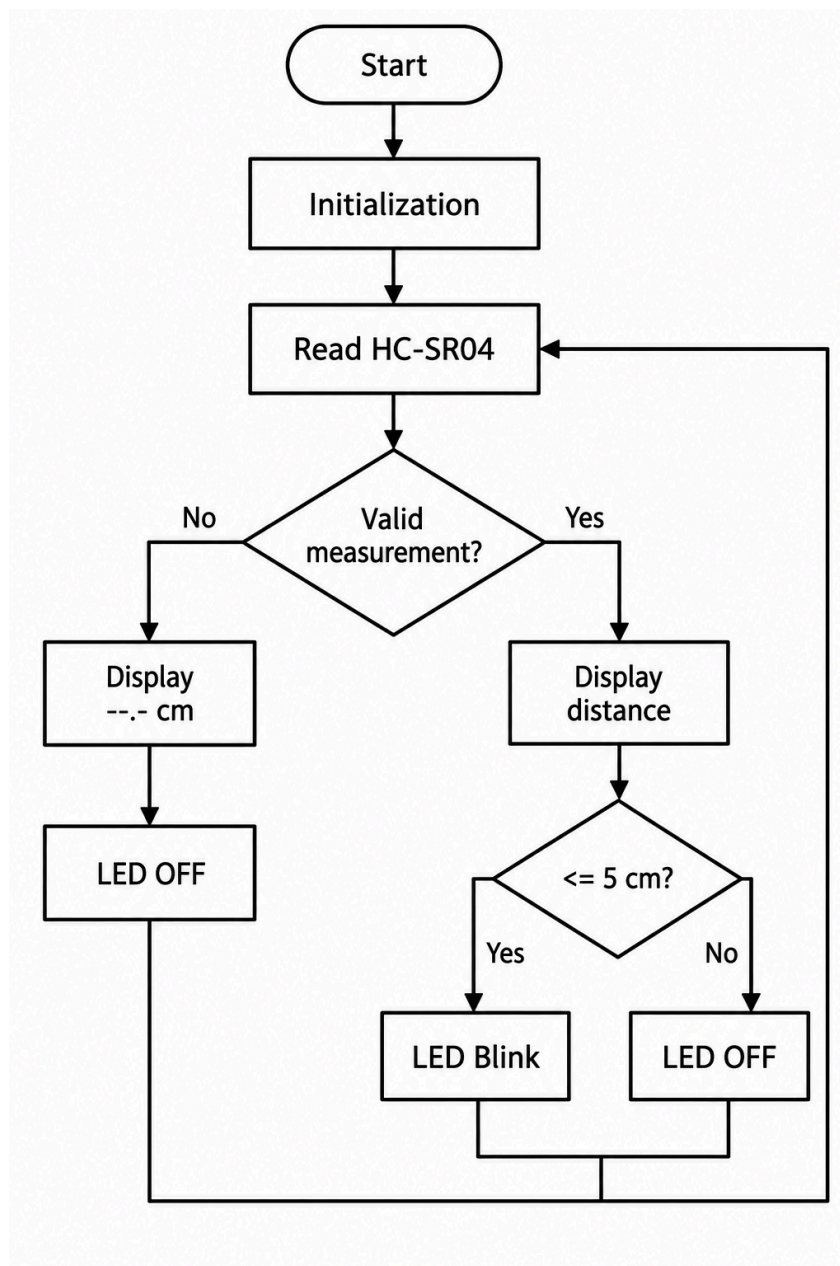


Figure 13.5.1. Flowchart of the distance monitoring program

5.4 Verification

Build and upload the program to the MaaZEDU Development Board. Place a reflective object in front of the HC-SR04 and vary its distance to verify the three main operating cases.

Condition	TFT Display	Red LED State
Distance > 5 cm	Distance value	OFF.
Distance ≤ 5 cm	Distance value	Blinks as a warning.
Invalid measurement	-- . - cm	OFF.

Table 13.5.1. Main verification cases for Lab 13

EXPECTED RESULT

The TFT continuously updates the measured distance. When the object is farther than 5 cm, the red LED remains OFF. When the object enters the range of 5 cm or less, the red LED blinks as a warning. If the sensor does not return a valid measurement, the TFT displays -- . - cm and the red LED remains OFF.

6 Extension

The main exercise uses only a distance value and one warning threshold to keep the program concise. From this structure, the application can be extended in several directions without changing the basic program architecture.

Status and color display: Add states such as `SAFE`, `WARNING`, and `DANGER` to the TFT. The text color or graphical elements can be changed for different distance ranges to make warning information more intuitive.

Multi-level warning: Replace the single threshold with multiple distance ranges or integrate the Passive Buzzer used in Lab 07. For example, the warning rate can change as an object moves closer to the sensor.

Integration with previously learned devices: Device APIs from earlier labs can be combined to build other small applications, for example:

- NTC + TFT + Buzzer for temperature monitoring and warning.
- RFID + TFT + LED / Buzzer to simulate an access control system.
- MPU6050 + TFT to display motion data or orientation status.
- HC-SR04 + Buzzer + TFT to develop a more expressive distance warning model.

These extensions do not require changes to the Device Layer. The application only needs to coordinate existing APIs and implement logic appropriate to the desired function.

7 References

- [1] New York University, *Lab: Ultrasonic Distance Sensor*, ITP Physical Computing.
- [2] SparkFun Electronics, *HC-SR04 Ultrasonic Sensor Datasheet*, Technical Datasheet.
- [3] MaaZEDU, *MaaZEDU Development Board Guide*, Rev. 1.0, 2025.
- [4] EduFramework, *Ultrasonic Device API*, EduFramework Source Code.
- [5] EduFramework, *LCD TFT Device API*, EduFramework Source Code.
- [6] EduFramework, *Digital I/O API and Pin Mapping*, EduFramework Source Code.